

[Fase 3]Proyecto Final

Integrantes:

- Carlos Andrés Rodríguez Pelayo
- Néstor Andrés Rivera Quintana
- Gerardo Josué Morales Sánchez
- Jared Vera Jaime

INTRODUCCION

Hemos llegado a la parte final del proyecto de la arquitectura MIPS de 32 bits. Es preciso recordar por lo que hemos pasado. Cada módulo y su implementación, tomando en cuenta sus entradas, salidas y participación directa dentro de la arquitectura de nuestro procesador, así como la participación que ha tenido la implementación de cada una de las tres instrucciones utilizadas (Tipos: R, I, J).

En esta precisa etapa del proyecto, añadimos la implementación de módulos para las instrucciones tipo J

La construcción de nuestro single datapath fue para una sola misión: correr un algoritmo previamente autorizado por el profesor para demostrar la correcta simulación del mismo.

En nuestro proyecto, hemos decidido usar el algoritmo de ordenamiento Bubble Sort. Este algoritmo consta de comparar pares de números, y el número más grande siempre “flotará” hacia la derecha, (de aquí recibe su nombre “burbuja”).

Como requerimiento para el presente reporte se nos pidió añadir cómo funcionaba el pipeline y buffers.

PIPELINE

El diseño de un procesador de 32 bits presenta una limitante de rendimiento, y es que el reloj debe de ser lo suficientemente lento para permitir que la instrucción sea cual fuese complete su camino a través de todos los componentes antes de que el reloj inicie la siguiente. Para optimizarlo, se introduce el pipelining. Es una segmentación, que divide la ejecución en cinco etapas:

IF (Instruction Fetch): Búsqueda de la instrucción en la memoria.

ID (Instruction Decode): Decodificación de la instrucción y lectura del banco de registros.

EX (Execute): Ejecución de la operación en la ALU o cálculo de direcciones.

MEM (Memory Access): Lectura o escritura en la memoria de datos.

WB (Write Back): Escritura del resultado de vuelta en el banco de registros.

Esto resulta en el hecho de que múltiples instrucciones pueden coexistir al mismo tiempo por distintas etapas sin que sus datos se pierdan o mezclen.

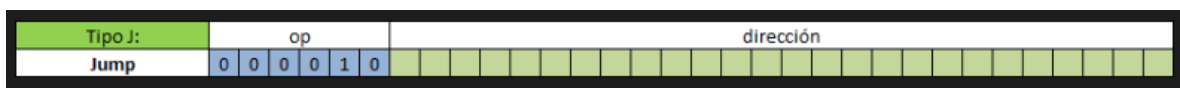
Una manera de verlo analógicamente es que actúan como barreras que atrapan y seccionan la información, de método que cada una de estas trabaja únicamente con la que contiene

Instrucciones tipo R

Es una operación de salto sin condición.

Su formato es de tipo J, definido en las estructuras básicas de instrucciones MIPS. Debido a esto, se compone de dos partes: Código de operación y Dirección.

El salto se realiza siempre a la dirección especificada por la instrucción.



La dirección corresponde a la dirección física de la Memoria de Instrucciones a la cual se debe realizar el salto.

Ejemplo:

Tipo J:	op						dirección																															
Jump	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1

Es un salto incondicional (JUMP) a la dirección 3 de la Memoria de Instrucciones.

Instrucción BRANCH

Es una instrucción de salto con condición. Se realiza el salto únicamente si la condición establecida se cumple.

Su formato es de tipo I, definido en las estructuras básicas de instrucciones MIPS. Debido a esto, se compone de 4 partes: Código de operación, rs, rt y Constante o Dirección.

Para evaluar la condición se compara el valor del registro rs contra el valor del registro rt.

Si la condición se cumple, entonces el valor de PC se debe actualizar a de la siguiente posición más el BranchAddr de la instrucción.

$PC = \text{Siguiete posición} + \text{BranchAddr}$

Existen dos tipos de instrucción BRANCH.

1. Branch Equals

Código de operación: 4 -> (000100)

rs: Numero del primer registro a comparar (5 bits)

rt: Numero del segundo registro a comparar (5 bits)

Constante o dirección: Valor a sumar.

Condición: ¿El valor almacenado en registro rs ES IGUAL al valor almacenado en registro rt?

Estructura instrucción BRANCH Equals Procesador MIPS Logisim

2. Branch NOT Equals

Código de operación: 5 (0000101)

rs: Numero del primer registro a comparar (5 bits)

rt: Numero del segundo registro a comparar (5 bits)

Constante o dirección: Valor a sumar.

Condición: ¿El valor almacenado en registro rs NO ES IGUAL al valor almacenado en registro rt?

Tipo I:	op	rs	rt	BranchAddr
Bne	0 0 0 1 0 1			

Ejemplo Branch.

Es una instrucción Branch Equals para saltar a la siguiente dirección en memoria + 4 si se cumple la condición que el contenido del registro 1 sea igual al contenido del registro 2.

Tipo I:	op	rs	rt	BranchAddr
Beq	0 0 0 1 0 0	0 0 0 0 1	0 0 0 1 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0

Algoritmo Bubble Sort

Este fue el algoritmo que decidimos hacer.

El ordenamiento de burbuja (Bubble Sort) es un algoritmo de comparación iterativo. Dada una lista de \$n\$ elementos, el algoritmo realiza un máximo de \$n-1\$ pasadas. En cada pasada, se comparan elementos adyacentes de la posición 0 hasta la posición \$n-1-i\$ (donde \$i\$ es el número de la pasada actual). Matemáticamente, el número total de comparaciones en el peor de los casos está determinado por la sumatoria:

$$\sum_{i=1}^{n-1} (n - i) = \frac{n(n - 1)}{2}$$

Para un arreglo de \$n = 9\$ elementos (como el de este proyecto), el número máximo de comparaciones se obtiene sustituyendo \$n=9\$.

$$\frac{9(9-1)}{2} = \frac{72}{2} = 36 \text{ comparaciones}$$

Como explicación/introducción para comprender su funcionamiento tenemos esta descripción, seguido del pseudocódigo del algoritmo.

2. Bubble Sort

Bubble sort is a sorting algorithm, which is commonly used in computer science. It is a simple comparison-based sorting algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order. Bubble sort is not suitable for large data sets as its average and worst case complexities are $O(n^2)$ where n is the number of items. We will discuss the complexity of bubble sort in detail.

Procedure

The procedure of the algorithm is described in following steps:

- Calling "BUBBLE_SORT" function, passing the array and its size as parameters.
- Repeat the following two steps i times where i is equal to $n-1$ and n is the size of the array.
- Starting from the first element of the array till j , repeat the following step where j is equal to $n-i-1$.
- Compare the current element with next element of the array. If the current element is greater than the next element of the array swap them.
- Return from "BUBBLE_SORT" function.

Pseudo code

function BUBBLE_SORT (array, size)

```

1. If size > 1 then:
2.   var swapped, iteration, index
3.   for iteration: 0 to size-1 do:
4.     swapped=false
5.     for index:0 to size-i-1 do:
6.       If array[index]>array[index+1] do:
7.         Swap array [index] and array [index+1]
8.         swapped=true
9.       End if
10.    End for

```

32 *Sorting using a combination of Bubble Sort, Selection Sort & Counting Sort*

```

11.      If swapped is false
12.        Return array
13.      End if
14.    End for
15.  End if
16. Return array

```

2019).

(Kumar & Singla,

OBJETIVO

Diseñar, implementar y validar un procesador MIPS de 32 bits utilizando el lenguaje de descripción de Hardware verilog, asegurando el correcto funcionamiento de las instrucciones tipo R, I & J. Demostrando conocer la manera en la que funciona el procesador en cada módulo, logrando implementar el algoritmo Bubble Sort.

OBJETIVOS ESPECIFICOS

Implementar el pipeline de 5 etapas: Integrar registros intermedios (IF/ID, ID/EX, EX/MEM, MEM/WB) para habilitar la ejecución segmentada y paralela de instrucciones.

Extender el repertorio de instrucciones: Adaptar la Unidad de Control y el datapath para soportar saltos incondicionales (instrucciones j y jal).

Desarrollar el algoritmo Bubble Sort: Programar el ordenamiento en ensamblador MIPS optimizando recursos mediante sumas y desplazamientos, sin utilizar multiplicadores hardware.

Gestionar señales de control: Definir la lógica de los multiplexores y la ALU para asegurar el flujo correcto entre las instrucciones de tipo R, I y J.

Validar mediante simulación: Verificar la integridad de los datos y el comportamiento del procesador en ModelSim, documentando cambios en el PC y acceso a memoria.

DESARROLLO

Acabamos de terminar de hablar de los buffers y pipelines, comenzaremos la parte descriptiva de dichos módulos.

IF/ID. Una vez el PC indicó la instrucción a leer, el buffer la captura. Su trabajo principal es sostener la instrucción el tiempo suficiente para que la unidad de control lea el Opcode y haga lo siguiente, como buscar los valores en el banco de registros.

ID/EX. Este buffer recoleta señales generadas por la Unidad de control, como ALUOp, MemWrite etc, los datos que sacó el banco de registros, y los junta.

EX/MEM. Una vez la ALU terminó su operación aritmética, este buffer captura ese resultado.

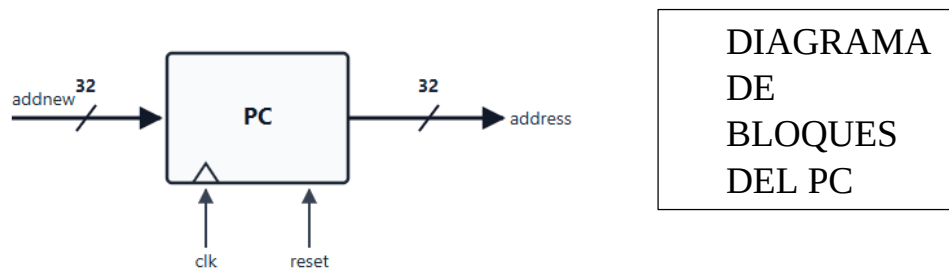
MEM/WB. Este buffer en resumidas cuentas guarda el valor final, recordemos que en las instrucciones siempre se elige un registro destino, se guarda hasta este momento y se guarda de vuelta en el banco de registros, cerrando el ciclo de la instrucción.

Otros módulos:

- PC.

Program Counter. Registro de 32 bits que como su nombre lo

indica funciona como contador de programa su función es almacenar la dirección de la instrucción a realizar. Está directamente sincronizada con el reloj.

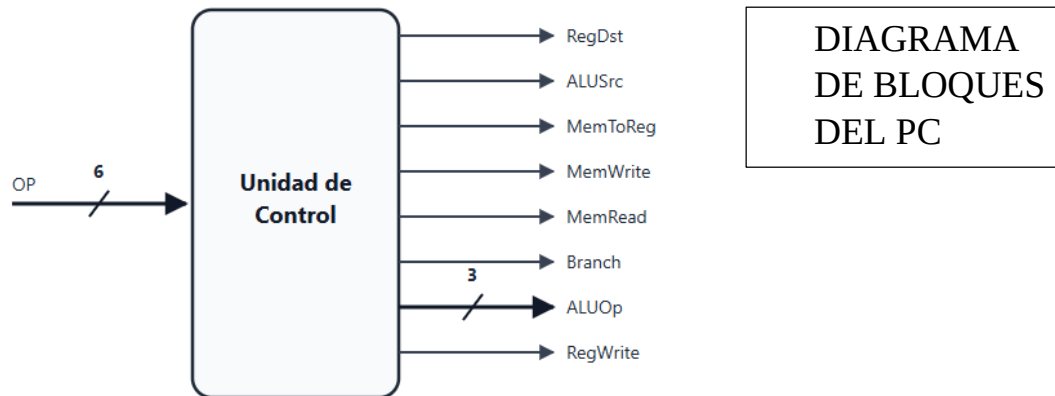


- Banco de registros.

Memoria interna del trabajo del procesador, compuesta por un arreglo de 32 bits x 32 bits. En nuestro proyecto esta memoria se inicializa leyendo un archivo externo que fue provisto por el profesor.

- UC.

Unidad de control. De manera análoga, es el cerebro, dirige los datos del datapath. Recibe los 6 bits más significativos independientemente de la instrucción que esté realizando (Tipo R, I, J). Se tienen casos definidos para las instrucciones de cualquier tipo y se pueden realizar dependiendo de la necesidad actual del programa/procesador.



- **ALUcontrol.** Es un decodificador (tipo switch) que le dice a la ALU que operación matemática o lógica realizar con exactitud.

Da el caso especial de que, por ejemplo, en las instrucciones tipo R, este módulo entra en un “sub-caso” y evalúa los 6 bits del campo funct (los bits menos significativos de los 32 que entran), esto es lo que le ayuda a determinar la operación a realizar. Esto sucede porque recordemos que en las tipo R, el Opcode, a pesar de que existe y abarca los 6 bits más significativos, siempre son ceros todos.

- **Extensor de signo (Sign Extend).**

Las instrucciones tipo I (Inmediatas), tienen un campo de 16 bits llamado Immediate, pero al ser arquitectura de 32 bits, la ALU y los registros operan con esta cantidad. Aquí entra la función del sign extend:

Toma los 16 bits de menor peso de la instrucción y los transforma en una palabra de 32 bits. Para hacerlo, replica el bit de signo (bit 15 de immediate) en las 16 posiciones superiores hasta completar 32.

- **Desplazador (Shift left 2)**

Este módulo realiza un desplazamiento lógico a la izquierda de 2 bits, lo que matemáticamente equivale a multiplicar el valor por 4.

Se utiliza específicamente para la instrucción de salto condicional beq (branch on equal). En MIPS, el immediate guarda un desplazamiento medido en palabras, pero el PC (Program counter) cuenta en bytes, entonces, al multiplicar el immediate extendido por 4, el desplazador convierte ese valor en el número exacto de bytes para que el PC use esa información y sepa cuanto saltar en la memoria.

- Multiplexores (Mux 2 a 1)

Estos ya habían sido implementados anteriormente, pero en esta ocasión, al convivir las instrucciones tipo R y tipo I, los caminos de datos se cruzan y se necesita saber cuál dejar pasar, decisiones tomadas por la unidad de control.

Por ejemplo. El mux de la ALU selecciona si el segundo operando de la ALU proviene del banco de registros (instrucciones tipo R) o del sign extend (para las tipo I).

- IMEM

Funciona como la memoria ROM (Read only memory). Es el lugar en donde se almacena el programa (el código ensamblador ya traducido a lenguaje máquina) que se va a ejecutar)

- MEM

Se dedica exclusivamente a guardar variables y datos generados durante la ejecución del programa (usadas por las instrucciones lw y sw).

Se puede escribir y leer de ella, dependiendo de las banderas MemToRead y MemToWrite.

- MUX2_1.

Multiplexor 2 a 1. Funciona como un switch, decide de donde proviene el dato y deja pasar uno u otro.

- Add32.

Su función es estar sumándole 4 al valor actual del PC, esto porque las instrucciones avanzan de 4 en 4 bytes.

- Branch_Ader

Parecido al anterior lo podemos notar en su nombre, este módulo funciona para cuando hay instrucciones de salto. Agarra ese PC+4 indicado anteriormente y le suma el número desplazado que saca el módulo shift left2. Así obtenemos la dirección exacta a la que tiene que brincar el programa si se dan las condiciones de salto.

- DPTR

El módulo DPTR es la parte principal del procesador, ya que se encarga de conectar y coordinar todos los componentes del pipeline. Su función es hacer que las instrucciones pasen correctamente por cada etapa del procesador: búsqueda, decodificación, ejecución, acceso a memoria y escritura de resultados.

Primero, el DPTR obtiene la instrucción desde la memoria utilizando el contador de programa (PC), el cual indica la dirección actual de la instrucción. Después, la instrucción pasa a la etapa de decodificación, donde la Unidad de Control identifica qué operación se debe realizar y el Banco de Registros proporciona los datos necesarios.

Más adelante, en la etapa de ejecución, la ALU realiza operaciones aritméticas o lógicas como sumas, restas, comparaciones y operaciones AND/OR. También se calculan las direcciones para instrucciones de salto y branch. Posteriormente, si la instrucción necesita acceder a memoria, el DPTR controla la lectura o escritura de datos. Finalmente, el resultado obtenido se guarda nuevamente en los registros correspondientes.

Además, el DPTR incluye registros intermedios entre cada etapa del pipeline, lo que permite que varias instrucciones se ejecuten al mismo tiempo de manera ordenada y eficiente. Gracias a esto, el procesador puede trabajar más rápido y aprovechar mejor cada ciclo de reloj. (En este último punto cobra sentido la explicación que se dio en la introducción, que talvez quedaría entendido en su momento, pero aquí se explica dentro de la práctica del módulo.

- ALU

La ALU es la calculadora principal del procesador. Su función es recibir dos datos de 32 bits y ejecutar una operación (matemática o lógica), pero

su rol cambia radicalmente dependiendo del tipo de instrucción que estés ejecutando:

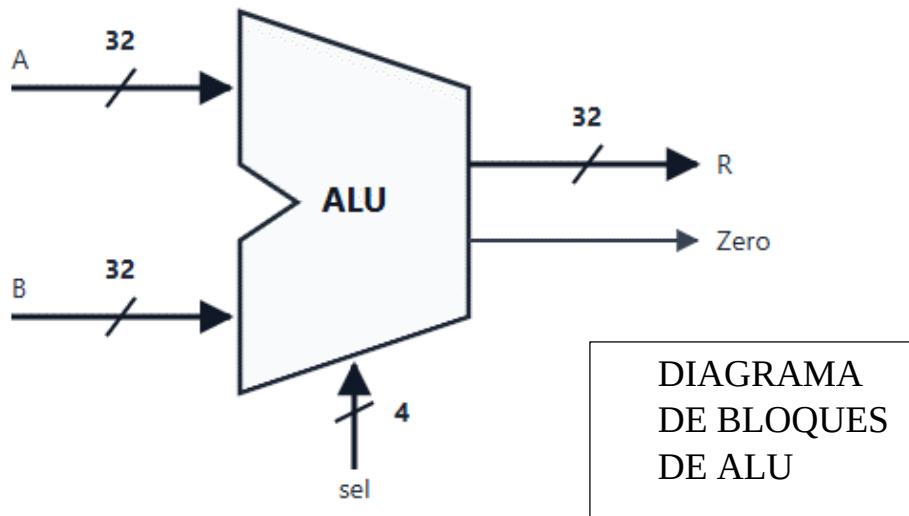
Instrucciones Tipo R. La ALU toma el contenido de dos registros y realiza la operación que determina el campo funct de la instrucción. El resultado se manda de vuelta al BR.

Instrucciones Tipo I. En este formato, la ALU cumple con dos trabajos:

Cargas y almacenamiento. Realiza lw y sw

Saltos condicionales. Realiza beq, bne.

Instrucciones tipo J. La ALU no se utiliza en estas instrucciones (como j o jal). El cálculo de la nueva dirección a la que debe saltar el PC se hace uniendo bits y desplazándolos en el datapath, puenteando por completo a la ALU.



BUBBLE SORT ALGORITMO

Algoritmo realizado en ensamblador.

Al inicio se le indica al procesador donde empieza la lista de números en memoria. Utilizamos un addi (el primero que se observa en el código, para decir que arrancamos en la posición cero, guardado en \$s0). Después, con lw traemos el tamaño total del arreglo (que es 9 en este caso).

Aquí entra una parte matemática muy importante del bubble Sort, y eso se puede ver ejemplificado en la línea de la foto siguiente:

```
addi $s2, $s1, -1
```

Se le resta 1 al tamaño total para asegurar de no salirnos de la lista, y ese límite como se puede observar se guarda en \$s2. Posteriormente se pone un contador en 0 (\$t0) que nos servirá para saber cuántas pasadas lleva la lista.

Entramos a la parte del bucle externo. Decidimos si seguimos ordenando o si terminamos usando slt y beq, dado el caso hipotético y con fines de explicación de que, si aún falta hacer pasadas, hacemos una sub para calcular hasta donde tiene que llegar el ciclo interno en la presente vuelta, ya que en cada pasada de este algoritmo en número más grande de la n-esima pasada ya fue acomodado hasta el final, entonces se tienen que revisar menos números en cada vuelta. Reiniciamos el contador (\$t1) y empezamos de nuevo.

En el bucle interno hace la parte más “trabajosa”. Después de verificar que no nos hayamos pasado del límite del arreglo, utilizamos el conocimiento de saber que la memoria avanza de 4 en 4 bytes; multiplicamos nuestro índice x 4 para saber que casilla de la memoria leer a continuación, y en lugar de realizar una multiplicación (recordando que el profesor prohibió dichas operaciones para el algoritmo), sumamos el índice consumo mismo dos veces usando add. Al sumarle eso a nuestra posición inicial (recordando que es \$s0), obtenemos la dirección de memoria exacta en donde estamos actualmente.

Se explicó la parte que de cierta manera pudiéramos haber desarrollado los conocimientos/competencias de semestres anteriores por los ciclos iterativos conocidos. Ahora, toca describir un poquito las operaciones lw de la manera que sigue: Usamos dos lw para sacar de memoria el número en donde estamos actualmente y el número que sigue inmediatamente. Ahora, usando slt y beq evaluamos el par y comparamos. Si ya están ordenados por tamaño el código salta y los deja en su lugar, pero si están al revés, usamos dos instrucciones sw para meterlos en la memoria, pero de manera alrevesado.

CONCLUSION

Conclusión de Gerardo Morales

A consideración que es el último entregable del semestre, vale la pena mencionar el arduo trabajo que se tuvo que hacer para conseguir los conocimientos necesarios y posteriormente la implementación de ellos para la realización. El conocimiento más allá de la actividad presente, es la sumatoria de actividades que se ha tenido que realizar para llegar al nivel visto en este punto.

Creo firmemente que el desarrollo de esta actividad ha sido probablemente uno de los retos más difíciles que he enfrentado en la carrera, pienso que esto viene desde la base de la complejidad que esto necesita. Pongamos este panorama: Para una persona que no está familiarizada con las ciencias computacionales le costará trabajo entender a priori desde lo más básico (hasta que constituya una sumatoria de habilidades y conocimientos que iguallen el nivel de comprensión que el individuo necesita para entender cosas más y más complicadas). Ahora, esto a nivel algorítmico o de programación puede significar un reto, y dependiendo de cada persona puede o no ser un reto que esta considere desafiante.

Con este tipo de actividades, pienso que se necesita un nivel de entendimiento más profundo, pues ya no es solo escribir con el teclado y darle click al botón de compilación, si no es, entender a nivel ingeniería, como funciona un procesador desde su estructura más básica.

Considero sorprendente el cómo cada uno de nosotros en sociedad podemos tener acceso a múltiples aparatos electrónicos que cuentan con procesadores (de distintas arquitecturas, pero, al fin y al cabo, procesadores), que realizan cálculos dentro de un hardware que corre a su vez un software realizado por el humano y que nos permite tener avances tecnológicos tan importantes. Pensamos que esta perspectiva de vida es indispensable como ingenieros, y, a fin de cuentas, como estudiantes de esta rama, es increíble saber cómo funciona a nivel tan profundo lo ya comentado.

Conclusión de Jared Vera

En conclusión, este proyecto fue uno de los más difíciles que he realizado, ya que tuve que aprender muchas cosas nuevas como Verilog, Python,

ensamblador, GitHub y trabajar en equipo. Durante el proceso hubo varios errores y problemas que resolver, pero eso también ayudó a entender mejor cómo funciona realmente un procesador y cómo se comunican sus diferentes partes.

Aunque fue complicado y tomó bastante tiempo, también fue una experiencia muy útil porque permitió aplicar conocimientos de programación y hardware en un proyecto más real. Además, ayudó a desarrollar paciencia, lógica y habilidades para resolver problemas. Sin duda, fue un proyecto retador, pero también una experiencia muy valiosa para mi aprendizaje.

Conclusión de Carlos Rodríguez

Sin duda fue un proyecto de los más desafiantes en mi carrera como estudiante, el hecho de contar con muchos nombres de variables, usar un lenguaje como python, ver código ensamblador, usar github, organizarse en equipo, etc. Fueron muchos factores que fueron añadiendo dificultad, pero a la vez, me permitió comprender como funcionan realmente las cosas que usamos de manera diaria, me parece sorprendente ver cómo con tan solo pulsos eléctricos (bits) se logra construir toda una arquitectura que a su vez construye cosas más y más grandes. El negocio del hardware me pareció desafiante y una experiencia que creo no es para mí, más sin embargo solo con este tipo de experiencias pude ser capaz de determinar ese resultado.

Conclusión de Néstor Rivera

En conclusión, al elaborar este proyecto sentí que realmente estaba haciendo un desafío, pues aprendí demasiadas cosas en muy poco tiempo. Toda la información vino de golpe y esto ocasionó que hubiera algunos problemas al momento de elaborar cada fase del respectivo proyecto.

Quisiera hacer énfasis en lo mucho que me ayudo este proyecto en agrandar mi conocimiento sobre todo en el ámbito de todo lo que pasa dentro de una computadora y como es que funciona toda esta arquitectura.

BIBLIOGRAFIA

Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design MIPS Edition: The Hardware/Software Interface* (6th ed.). Morgan Kaufmann.

Harris, D. M., & Harris, S. L. (2012). *Digital Design and Computer Architecture (MIPS Edition)* (2nd ed.). Morgan Kaufmann.

Kalamazoo College. (s.f.). MIPS Instruction Formats. Recuperado el 23 de mayo de 2026, de <http://www.cs.kzoo.edu/cs230/Resources/MIPS/MachineXL/InstructionFormats.html>

Mendoza, L. (2021, 7 de mayo). Instrucciones JUMP y BRANCH. Procesador MIPS Logisim. <https://procesadormipslogisim.blogspot.com/2014/02/estructura-de-instrucciones-jump-y-branch.html>

Kumar, S., & Singla, P. (2019). Sorting using a combination of Bubble Sort, Selection Sort & Counting Sort. *International Journal of Mathematical Sciences and Computing*, 5(2), 30–43. <https://doi.org/10.5815/ijmsc.2019.02.03>